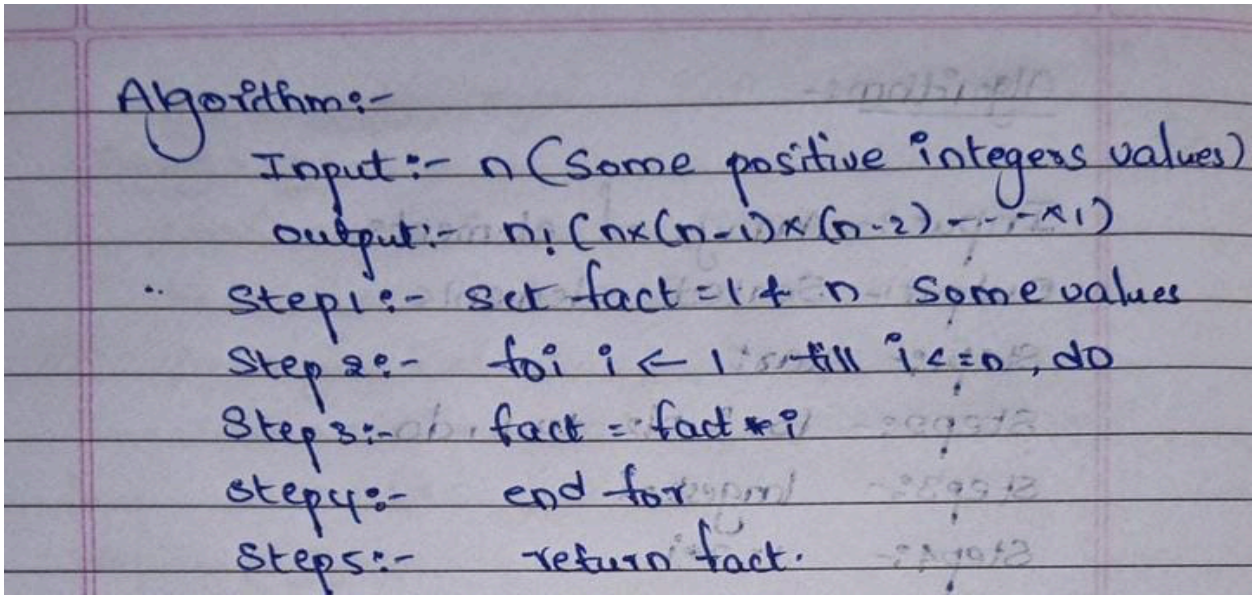
PRACTICAL-4

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

Factorial Using Iterative Method

ALGORITHM:

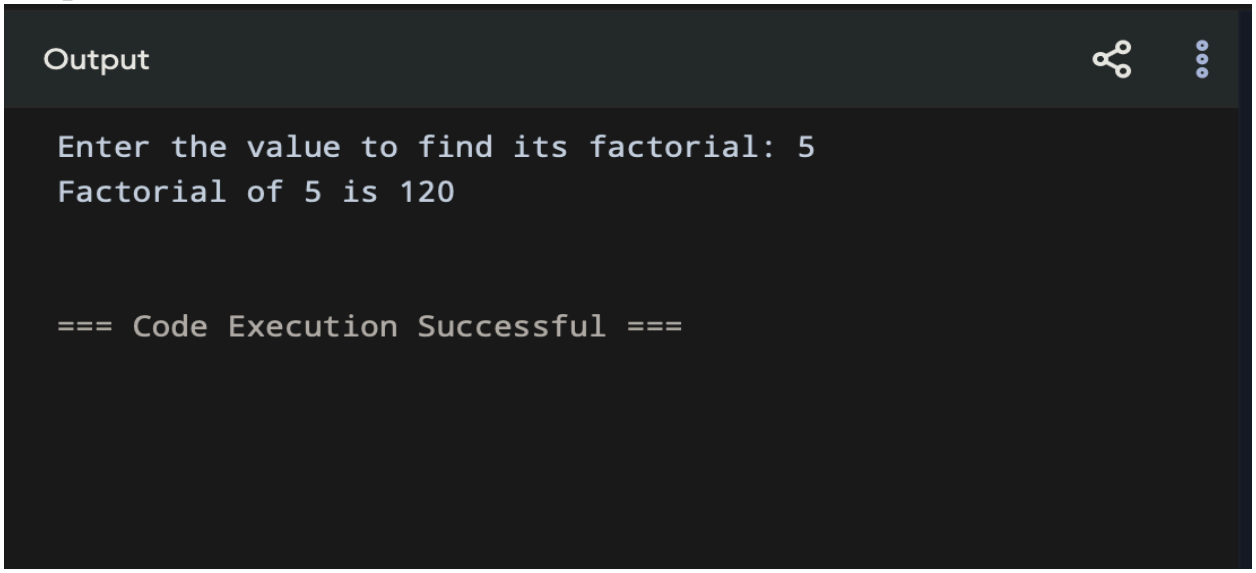


CODE:

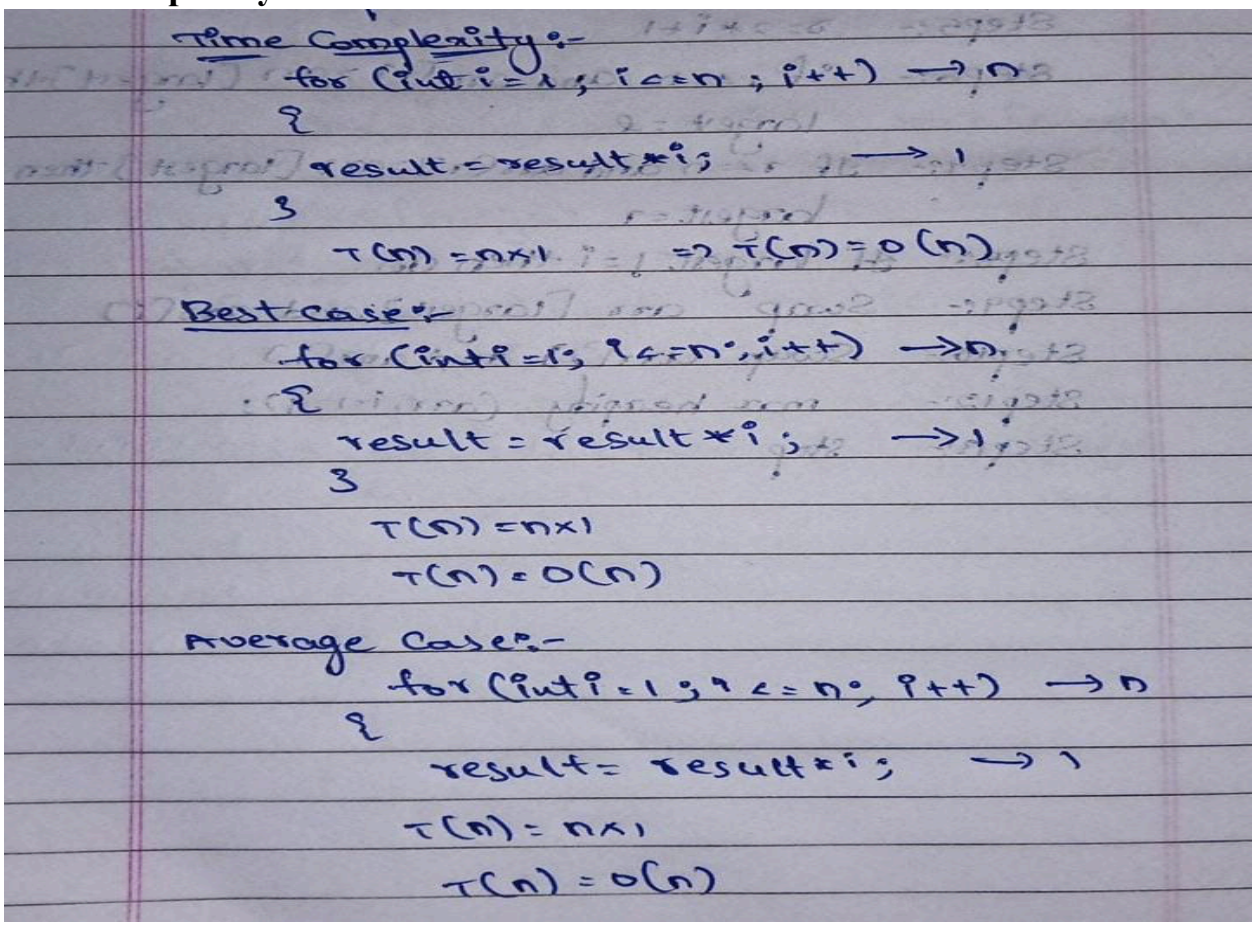
```
#include <iostream>
using namespace std;

class Factorial
{
public:
    int fact(int n)
    {
        int result = 1;

        for (int i = 1; i <= n; i++)
        {
            result = result * i;
        }
```

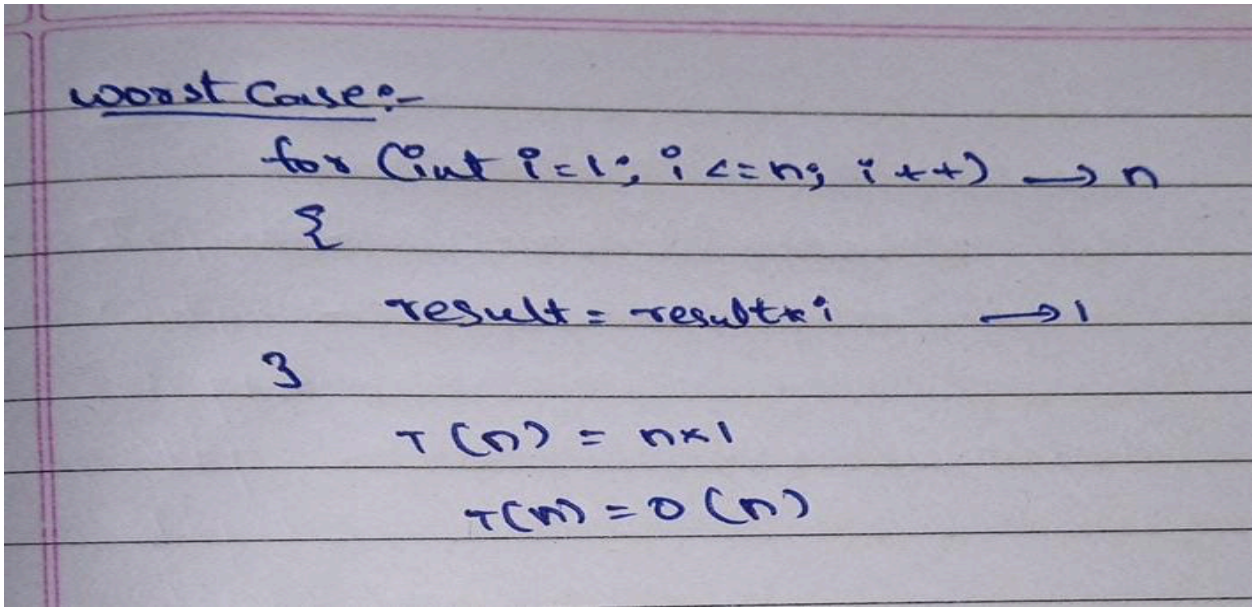
```
        return result;
    }
};

int main()
{
    int n,result;
    Factorial obj;


    cout << "Enter the value to find its factorial: ";
    cin >> n;


    result = obj.fact(n);

    cout << "Factorial of " << n << " is " << result << endl;

    return 0;
}
```

Output:



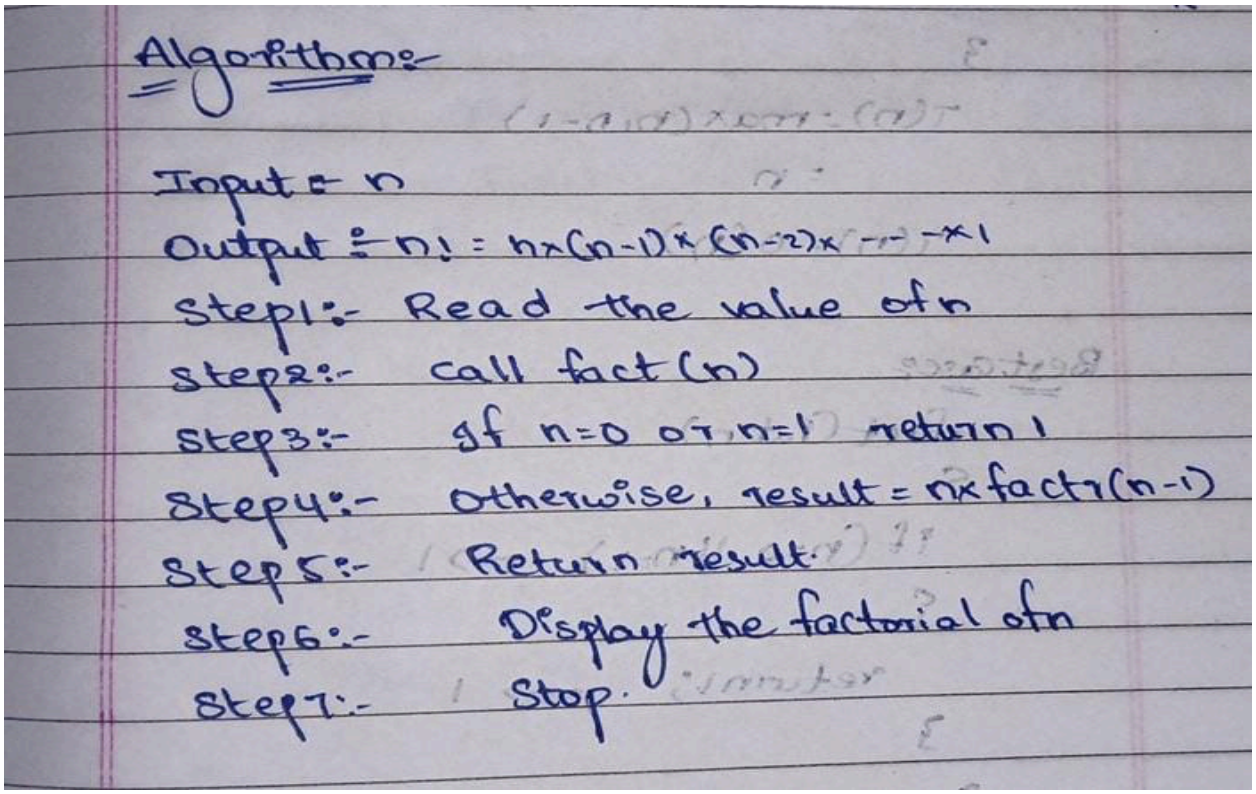
Time Complexity:





Factorial Using Recursive Method:

Algorithm



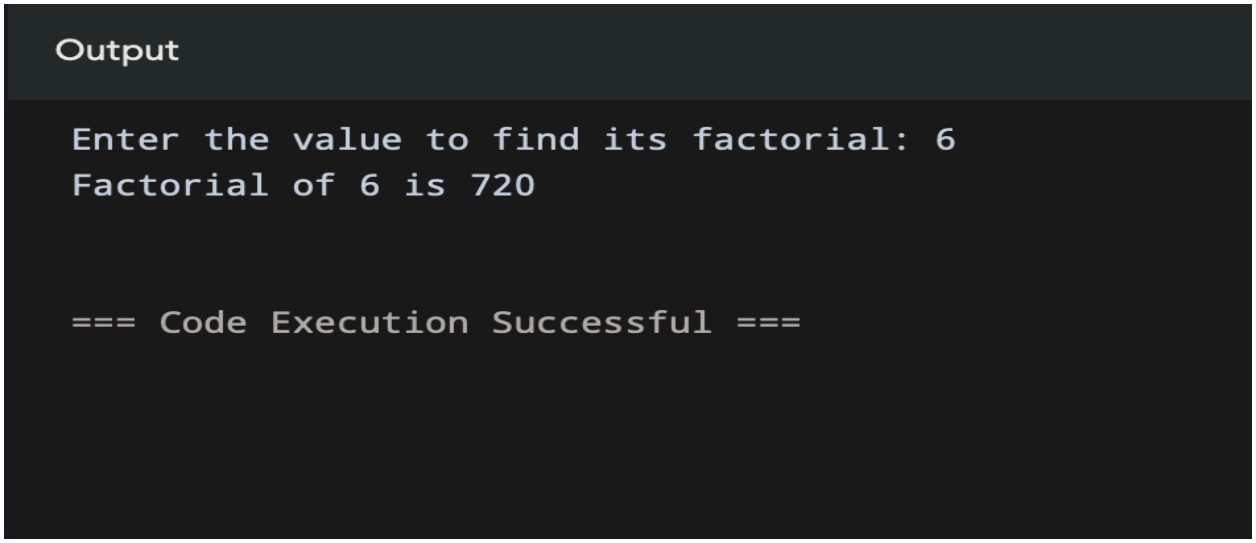
Code:

```
#include <iostream>

using namespace std;

class Factorial

{

public:

    int factr(int n)
```

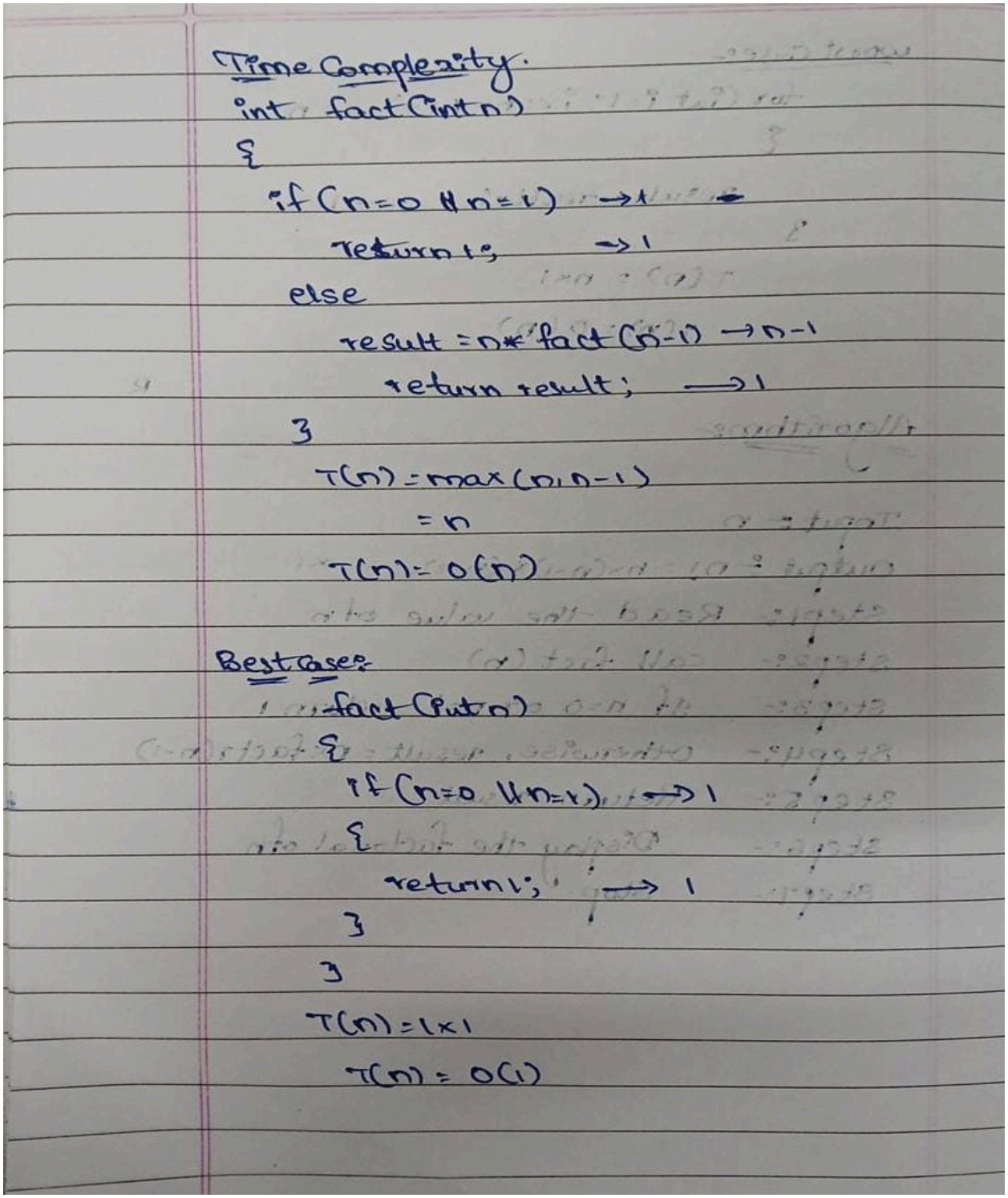
```
    {
        if (n == 0 || n == 1) {
            return 1;
        }
        else
        {
            int result = n * factr(n -
1);
            return result;
        }
    }
};
int main()
{
    int n, result;
    // Create an object of the class
    Factorial obj;
    cout << "Enter the value to
find its factorial: ";
```

```
    cin >> n;

    result = obj.factr(n);

    cout << "Factorial of " << n
<< " is " << result << endl;

    return 0;

}
```

Output:



TIME COMPLEXITY:



Time Complexity.

```
int fact(int n)
{
  if(n==0 || n==1)  →1
    return 1;       →1
  else
    result = n*fact(n-1)  →n-1
      return result;      →1
}
```

$T(n) = \max(n, n-1)$

$= n$

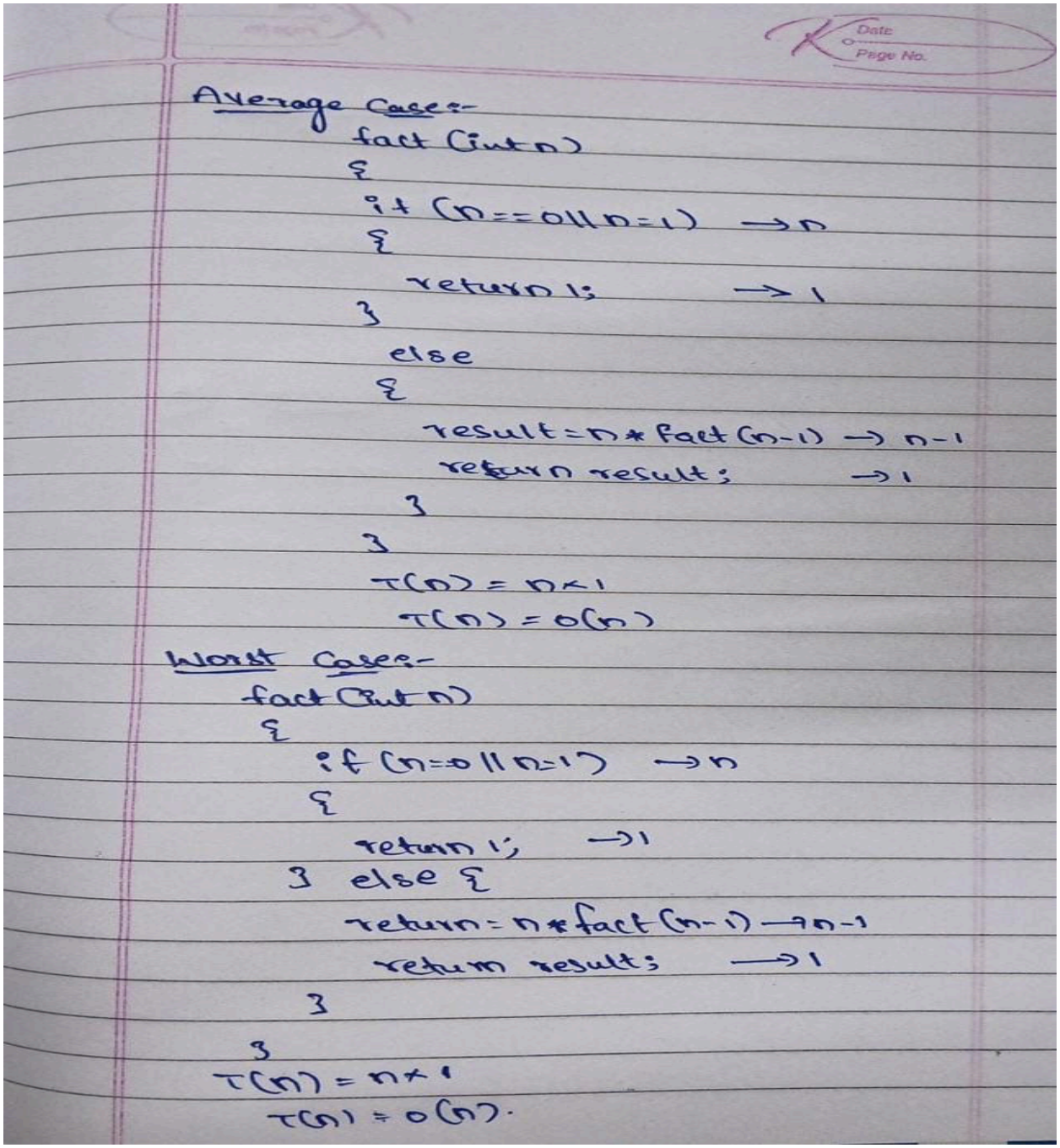
$T(n) = O(n)$

Best Case:

```
fact(int n)
{
  if(n==0 || n==1)  →1
  {
    return 1;       →1
  }
}
```

$T(n) = 1 \times 1$

$T(n) = O(1)$



Average Case:-

```
fact (int n)
{
 if (n==0||n=1)  →n
 {
   return 1;      →1
 }
  else
  {
    result=n*fact(n-1) → n-1
    return result;     →1
  }
}
T(n) = n+1
T(n) = O(n)
```

Worst Case:-

```
fact(int n)
{
  if (n=0||n=1)  →n
  {
    return 1;    →1
  } else {
    return=n*fact(n-1) →n-1
    return result;  →1
  }
}
T(n) = n+1
T(n) = O(n).
```

Conclusion:

The factorial program was successfully implemented using both iterative and recursive methods. Both methods produce the same result, but the recursive method uses function calls and stack space. The time complexity of both methods is O(n), making them efficient for calculating factorials.